

Khoa KHMT – Trường CNTT&TT, Đại học Cần Thơ**Môn học: CT332 Trí tuệ nhân tạo****Bài thực hành giải thuật di truyền Genetic Algorithm**

Mục tiêu:

- Sinh viên hiểu về cách mô hình hoá bài toán theo giải thuật di truyền.
- Làm quen với 2 thư viện phổ biến DEAP và PyGAD
- So sánh ưu và nhược điểm của 2 thư viện DEAP và PyGAD.
- Cài đặt các thành phần cơ bản và vẽ kết quả.
- Bài thực hành có sử dụng 1 số kiến thức từ aivietnam.edu.vn

1. Giới thiệu**1.1 Bảng từ khoá/khái niệm**

Từ khoá	Ý nghĩa
Genetic Algorithm	Giải thuật di truyền: phương pháp tối ưu hoá theo quần thể, mô phỏng tiến hóa tự nhiên qua các bước chọn lọc, lai ghép và đột biến.
Population	Quần thể: tập các cá thể ở một thế hệ.
Individual	Cá thể: một lời giải ứng viên; trong One-Max là chuỗi nhị phân.
Gene	Gen: cách biểu diễn của cá thể; vd: trong One-Max là một bit 0/1, trong bài so sánh chuỗi thì gen là 1 chuỗi độ dài tương ứng ...
Generation	Thế hệ: một vòng lặp tiến hóa của quần thể.
Initialization	Khởi tạo quần thể: tạo quần thể ban đầu (thường ngẫu nhiên).
Fitness	Độ thích nghi: điểm đánh giá chất lượng cá thể (càng cao càng tốt).
Fitness function	Hàm thích nghi: hàm tính fitness cho cá thể; One-Max: tổng số bit 1.
Fitness evaluation	Đánh giá độ thích nghi: bước chấm điểm tất cả cá thể (dựa vào chỉ số fitness).
Selection	Chọn lọc: chọn cá thể tốt vào nhóm để lai ghép/đột biến.
Selection Pool	Nhóm chọn lọc: tập các cá thể được giữ lại để tạo thế hệ con.
(Tournament) Selection	Cơ chế chọn lọc tương ứng ví dụ trong bài: rút ngẫu nhiên 2 cá thể, giữ cá thể có fitness cao hơn.
Crossover	Lai ghép: kết hợp gene của hai cá thể cha mẹ để tạo cá thể con.
Uniform Crossover	Lai ghép đồng nhất: tại mỗi vị trí gene, quyết định lấy gene từ cha hay mẹ theo một xác suất cho trước, gọi là “tỉ lệ hoán đổi”.
Mutation	Đột biến: lật bit với xác suất nhỏ để tăng đa dạng và tránh bị kẹt ở cực trị cục bộ.
Crossover rate (p_{cross})	Tỉ lệ lai/hoán đổi: xác suất lấy gene từ cha/mẹ tại mỗi vị trí trong uniform crossover.
Mutation rate (p_{muta})	Tỉ lệ đột biến: xác suất lật bit tại mỗi gene.
Elitism	Cá thể ưu tú: giữ nguyên một số cá thể tốt nhất sang thế hệ tiếp theo.

1.2 Giới thiệu

Giải thuật di truyền dựa vào quá trình tiến hóa của sinh vật trong tự nhiên qua nhiều thế hệ. Hiểu một cách đơn giản, trong một quần thể sinh vật, các cá thể buộc phải tiến hóa liên tục qua từng thế hệ để thích nghi với môi trường và tồn tại.

Giải thuật di truyền lấy cảm hứng từ luật di truyền của Mendel (công bố 1865) hơn là dựa trên thuyết tiến hoá của Darwin

		pollen ♂	
		B	b
pistil ♀	B	BB	Bb
	b	Bb	bb

[Bảng Punnett](#) mô tả kết quả thí nghiệm của Mendel trên đậu Hà Lan khi cho F_1 tự thụ phấn. (nguồn : wikipedia)

1.3 Thành phần cơ bản của giải thuật

Khởi tạo (Initialization): Tạo một quần thể ban đầu gồm các cá thể ngẫu nhiên (mỗi cá thể đại diện cho một giải pháp).

Hàm thích nghi (Fitness Function): Đánh giá "độ tốt" của mỗi cá thể, cá thể có độ thích nghi cao hơn sẽ có khả năng được chọn cao hơn.

Chọn lọc (Selection): Chọn các cá thể tốt nhất từ thế hệ hiện tại để làm cha mẹ cho thế hệ sau.

Lai ghép (Crossover): Kết hợp các đặc điểm (gen) của hai cá thể cha mẹ để tạo ra con lai mới, hy vọng tạo ra giải pháp tốt hơn.

Đột biến (Mutation): Thay đổi nhỏ ngẫu nhiên trên các cá thể con để duy trì sự đa dạng di truyền và tránh mắc kẹt trong tối ưu cục bộ.

Lặp lại: Quá trình chọn lọc - lai ghép - đột biến lặp lại cho đến khi tìm được giải pháp tối ưu hoặc đạt đủ số thế hệ.

2 Thư viện sử dụng

DEAP và PyGAD đều là hai thư viện Python phổ biến dùng để triển khai giải thuật di truyền (Genetic Algorithm - GA)

Tiêu chí	DEAP (Distributed Evolutionary Algorithms in Python)	PyGAD	Nhận xét
Năm ra mắt	2010–2012, rất trưởng thành	2018–2020, trẻ hơn nhưng phát triển nhanh	DEAP lâu đời, ổn định hơn
Tính dễ sử dụng	Trung bình – Khó hơn (phải hiểu toolbox, creator, algorithms)	Rất dễ – API thiết kế class đơn giản	
Cú pháp điển hình	Dùng <code>creator</code> , <code>toolbox.register()</code> , <code>algorithms.eaSimple()</code>	<code>ga = pygad.GA(...)</code> → <code>ga.run()</code> → <code>ga.best_solution()</code>	
Mức độ tùy chỉnh	Rất cao – gần như viết lại thuật toán nếu muốn	Cao, nhưng trong khuôn khổ class GA	DEAP linh hoạt hơn cho nghiên cứu
Hỗ trợ loại bài toán	GA, ES, GP, PSO, CMA-ES, multi-objective (NSGA-II, SPEA2, ...), co-evolution	GA đơn mục tiêu + multi-objective, tối ưu mạng nơ ron (Keras & PyTorch),	DEAP đa dạng hơn
Tích hợp với học sâu (Deep Learning)	Không có sẵn – phải tự viết fitness function gọi model	Tích hợp sẵn Keras & PyTorch (tối ưu trọng số NN bằng GA)	PyGAD vượt trội ở khoản này
Trực quan hoá	Không có sẵn (phải tự vẽ bằng matplotlib + logbook)	Có sẵn plot fitness, giải pháp, ...	PyGAD tiện lợi hơn
Xử lý song song, phân tán	Hỗ trợ tốt (<code>multiprocessing</code> , <code>SCOOP</code> , <code>map</code>)	Hỗ trợ multiprocessing cơ bản, gene-space parallel	DEAP mạnh hơn ở xử lý song song phức tạp
Cộng đồng	Cộng đồng học thuật ổn định, vẫn được update	Tác giả rất tích cực (Ahmed Fawzy Gad), update thường xuyên	
Kích thước & Phụ thuộc	Nhẹ, phụ thuộc ít (numpy, matplotlib tùy chọn)	Nhẹ, phụ thuộc numpy, matplotlib, tqdm, hỗ trợ torch/keras tùy chọn	Tương đương
Hiệu suất	Tốt khi tối ưu, nhưng không nên lạm dụng toolbox	Nhanh hơn cho các bài toán thông thường nhờ thiết kế đơn giản	
Phù hợp nhất cho	Nghiên cứu, bài toán tiến hóa phức tạp, multi-objective, GP, ES	Prototyping nhanh, tối ưu NN bằng GA, dạy học, ứng dụng thực tế đơn giản	

2.1 Kiến trúc tổng thể của DEAP

DEAP được thiết kế theo triết lý "làm rõ ràng thuật toán và minh bạch cấu trúc dữ liệu" (explicit algorithms & transparent data structures), thay vì kiểu "black-box" như một số thư viện khác.

Các module cốt lõi (core architecture) bao gồm:

Module	Chức năng chính	Ví dụ
base	Chứa các lớp cơ bản (Fitness, Toolbox, ...) không có sẵn trong Python chuẩn	<code>base.Fitness</code> , <code>base.Toolbox</code>
creator	Meta-factory: tạo class động (dùng <code>create()</code>) cho Fitness, Individual, ...	<code>creator.create("FitnessMax", base.Fitness, weights=(1.0,))</code>
tools	Các hàm công cụ sẵn có (init, crossover, mutation, selection, statistics, ...)	<code>tools.initRepeat</code> , <code>tools.cxTwoPoint</code> , <code>tools.selTournament</code>
algorithms	Các thuật toán GA/ES/PSO. (eaSimple, eaMuPlusLambda, ...)	<code>algorithms.eaSimple</code> , <code>algorithms.varAnd</code>
gp	Genetic Programming	<code>gp.PrimitiveTree</code> , <code>gp.compile</code>
cma	Covariance Matrix Adaptation Evolution Strategy	<code>cma.Strategy</code>
benchmarks	Các bài test chuẩn (OneMax, Knapsack, Rastrigin, TSP, ...)	<code>benchmarks.rastrigin</code>
dtm / SCOOP	Hỗ trợ phân tán / song song (parallel evaluation)	

2.2 Thành phần cơ bản PyGAD

PyGAD là thư viện mã nguồn mở (open-source) của Python chuyên dùng để xây dựng **Giải thuật Di truyền (Genetic Algorithm)** và tối ưu hóa các mô hình học máy (đặc biệt là tối ưu trọng số Neural Network bằng GA).

Ưu điểm:

- Dễ sử dụng (API đơn giản kiểu class).
- Hỗ trợ **single-objective** và **multi-objective** (NSGA-II).
- Tích hợp sẵn với **Keras** và **PyTorch**.
- Có nhiều hàm vẽ biểu đồ đẹp (`plot_fitness`, `plot_genes...`).
- Linh hoạt cao nhờ fitness function tùy chỉnh.

1 số lớp/ module chính của PyGAD:

Module	Mục đích chính
pygad	Module cốt lõi – xây dựng và chạy Genetic Algorithm (lớp GA)
pygad.nn	Xây dựng mạng nơ ron thần kinh ảo -Artificial Neural Network (ANN) đơn giản từ đầu

Module	Mục đích chính
pygad.gann	Tối ưu Neural Network bằng GA (GANN = Genetic Algorithm + Neural Network)
pygad.cnn	Xây dựng Convolutional Neural Network (CNN)
pygad.gacnn	Tối ưu CNN bằng GA
pygad.kerasga	Tích hợp GA với mô hình Keras (rất phổ biến)
pygad.torchga	Tích hợp GA với mô hình PyTorch
pygad.utils	Các lớp hỗ trợ (Crossover, Mutation, ParentSelection, NSGA2...)

Module quan trọng nhất và được dùng nhiều nhất là pygad (chứa lớp GA).

```
ga_instance = pygad.GA(
    num_generations=50,
    num_parents_mating=10,
    fitness_func=fitness_func,
    sol_per_pop=300,
    num_genes=40,
    ...
)
```

2.2.1. Các tham số trong hàm xây dựng constructor (`__init__`)

a) Tham số bắt buộc:

- `num_generations`: Số thế hệ (số lần lặp).
- `num_parents_mating`: Số cá thể cha mẹ được chọn để lai tạo mỗi thế hệ.
- `fitness_func`: Hàm tính fitness (quan trọng) – nhận tham số (`ga_instance`, `solution`, `solution_idx`) và trả về giá trị số (càng lớn càng tốt).
- `sol_per_pop`: Kích thước quần thể (số cá thể mỗi thế hệ).
- `num_genes`: Số gene của mỗi cá thể (độ dài vector giải pháp).

b) Tham số tùy chọn:

- `gene_type`: Kiểu dữ liệu của gene (int, float, ...).
- `init_range_low` / `init_range_high`: Phạm vi khởi tạo gene ban đầu.
- `parent_selection_type`: "sss", "tournament", "rank", "rws", ...
- `crossover_type`: "single_point", "two_points", "uniform", "scattered", ...
- `mutation_type`: "random", "swap", "inversion", "scramble", ...
- `mutation_percent_genes`: Tỷ lệ % gene bị đột biến.
- `keep_parents`: Số cha mẹ tốt nhất giữ lại (elitism).
- `initial_population`: Cho phép truyền quần thể ban đầu (rất hữu ích).
- `on_generation`: Callback chạy sau mỗi thế hệ (dùng để in tiến trình, lưu log...).
- `on_fitness`, `on_parents`, `on_crossover`, `on_mutation`: Các callback khác.

c. Các phương thức (methods) quan trọng của `ga_instance`

- `run()` → Chạy toàn bộ quá trình tiến hóa.
- `best_solution()` → Trả về 3 giá trị: (`best_solution`, `best_fitness`, `best_solution_idx`).
- `plot_fitness()` → Vẽ biểu đồ fitness theo thế hệ (tốt nhất, trung bình...).
- `plot_genes()` → Vẽ sự thay đổi của từng gene qua các thế hệ.
- `plot_new_solution_rate()` → Tỷ lệ giải pháp mới mỗi thế hệ.
- `plot_result()` → Biểu đồ tổng hợp.
- `save()` / `load()` → Lưu và tải instance GA.
- `summary()` → In tóm tắt thông tin GA.

d. Các thành phần hỗ trợ khác

- **Callbacks:** Cho phép bạn can thiệp vào từng bước (`on_generation`, `on_mutation`...).
- **NSGA-II:** Hỗ trợ tối ưu đa mục tiêu (multi-objective).
- **Parallel fitness evaluation:** Có thể tính fitness song song để nhanh hơn với quần thể lớn.
- **Gene space:** Giới hạn giá trị cho từng gene (rất hữu ích khi giải quyết bài toán có ràng buộc).

3 Bài toán**Bài 1: Bài toán One-Max (Tìm chuỗi nhị phân tối ưu)**

Mô tả: Tìm chuỗi nhị phân dài 20 bit có tổng số bit 1 lớn nhất (tức là tất cả là 1). Đây là bài toán đơn giản để làm quen với các thành phần cơ bản của giải thuật di truyền.

Gợi ý:

- Định nghĩa kích thước quần thể (`POPULATION_SIZE = 100`), độ dài chuỗi (`GENES = 20`), số thế hệ tối đa (`MAX_GENERATIONS = 100`).
- tạo cá thể ngẫu nhiên: danh sách 20 số 0 hoặc 1.
- tính fitness: tổng các bit trong cá thể (càng lớn càng tốt).
- chọn lọc tournament (chọn 3 cá thể ngẫu nhiên, lấy cái tốt nhất).
- lai chéo đơn điểm (single-point crossover).
- đột biến (xác suất 1% lật bit 0↔1).
- Tạo quần thể ban đầu, lặp qua các thế hệ: chọn cha mẹ, lai chéo, đột biến, tạo quần thể mới.
- Theo dõi cá thể tốt nhất mỗi thế hệ và dừng sớm nếu đạt fitness tối đa.
- In kết quả cuối cùng.

```
import random
```

```

import matplotlib.pyplot as plt
from deap import base, creator, tools, algorithms

# Tạo fitness (maximize) và Individual
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

# Khởi tạo toolbox
toolbox = base.Toolbox()

# Đăng ký các hàm
toolbox.register("attr_bool", random.randint, 0, 1)
toolbox.register("individual", tools.initRepeat, creator.Individual,
toolbox.attr_bool, n=40)
toolbox.register("population", tools.initRepeat, list,
toolbox.individual)

# Hàm đánh giá
def evalOneMax(individual):
    return sum(individual),

toolbox.register("evaluate", evalOneMax)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=0.05)
toolbox.register("select", tools.selTournament, tournsize=3)

# Chạy thuật toán
def main():
    pop = toolbox.population(n=300)
    hof = tools.HallOfFame(1)
    stats = tools.Statistics(lambda ind: ind.fitness.values)
    stats.register("max", max)

    logbook = tools.Logbook()
    logbook.header = ["gen", "avg", "max", "min"]

    # Chạy eaSimple (thuật toán GA cơ bản)
    pop, logbook = algorithms.eaSimple(pop, toolbox, cxpb=0.5,
mutpb=0.2,
                                ngen=50, stats=stats,
halloffame=hof, verbose=True)

```

```
# Thêm stats để theo dõi
stats = tools.Statistics(key=lambda ind: ind.fitness.values[0])
stats.register("avg", lambda pop: sum(i.fitness.values[0] for i
in pop)/len(pop))
stats.register("max", lambda pop: max(i.fitness.values[0] for i
in pop))
stats.register("min", lambda pop: min(i.fitness.values[0] for i
in pop))

print("Chuỗi tốt nhất:", hof[0])
print("Số lượng 1:", sum(hof[0]))
return pop, logbook, hof

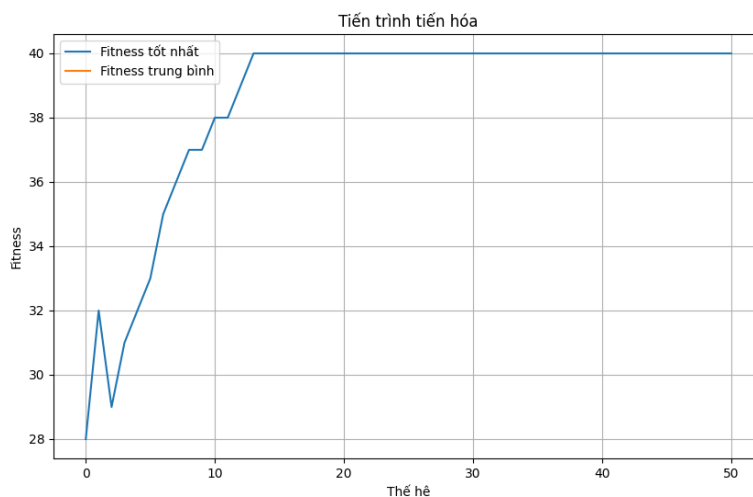
if __name__ == "__main__":
    main()
```

gen	nevals	max
0	300	(28.0,)
1	187	(32.0,)
2	204	(29.0,)
3	171	(31.0,)
4	169	(32.0,)
5	195	(33.0,)
6	188	(35.0,)
7	193	(36.0,)
8	181	(37.0,)
9	181	(37.0,)
10	186	(38.0,)
11	197	(38.0,)

[illegible]


```
# Vẽ biểu đồ
gen = logbook.select("gen")
fit_max = logbook.select("max")
fit_avg = logbook.select("avg")

plt.figure(figsize=(10,6))
plt.plot(gen, fit_max, label="Fitness tốt nhất")
plt.plot(gen, fit_avg, label="Fitness trung bình")
plt.xlabel("Thế hệ")
plt.ylabel("Fitness")
plt.title("Tiến trình tiến hóa")
plt.legend()
plt.grid(True)
plt.show()
```



```
import pygad
import numpy as np
import matplotlib.pyplot as plt

# =====
# 1. Định nghĩa hàm fitness
# =====
def fitness_func(ga_instance, solution, solution_idx):
    """
```

```

Fitness = số lượng bit 1 trong solution (chuỗi nhị phân).
PyGAD tối đa hóa fitness, nên giá trị càng lớn càng tốt.
"""

fitness = np.sum(solution)
return fitness

# =====
# 2. Các tham số cơ bản GA (pyGAD có nhiều tham số hơn DEAP)
# =====
num_generations = 50          # Số thế hệ
num_parents_mating = 10      # Số cha mẹ tham gia lai tạo mỗi thế
                               # hệ
sol_per_pop = 300            # Kích thước quần thể
num_genes = 40                # Độ dài chuỗi (số gene/bit)

# =====
# 3. Khởi tạo instance PyGAD
# =====
ga_instance = pygad.GA(
    num_generations=num_generations,
    num_parents_mating=num_parents_mating,
    fitness_func=fitness_func,
    sol_per_pop=sol_per_pop,
    num_genes=num_genes,
    save_solutions=True,      # Lưu tất cả giải pháp tốt nhất qua các
                               # thế hệ
    # để vẽ biểu đồ sau này (tổn bộ nhớ hơn)
    save_best_solutions=True, # Lưu giải pháp tốt nhất của mỗi thế
                               # hệ

    gene_type=int,           # Gene là số nguyên (0 hoặc 1 như đề bài)
    init_range_low=0,
    init_range_high=2,        # random.randint(0, 1)

    # Parent selection: steady-state selection (giống tournament phổ
    # biến)
    parent_selection_type="sss", # Hoặc "tournament", "rank",
    "rws",...

    # Giữ lại một phần cha mẹ tốt nhất (tương đương elitism trong
    # DEAP)
    keep_parents=2,

```

```

# Crossover: single-point (tương tự cxTwoPoint trong DEAP)
crossover_type="single_point",

# Mutation: random (flip bit với xác suất)
mutation_type="random",
mutation_percent_genes=10,    # ~10% gene bị đột biến mỗi cá thể
crossover_probability=0.8,
)

# =====
# 4. Chạy thuật toán
# =====
print("Bắt đầu giải One-Max bằng PyGAD...")
ga_instance.run()

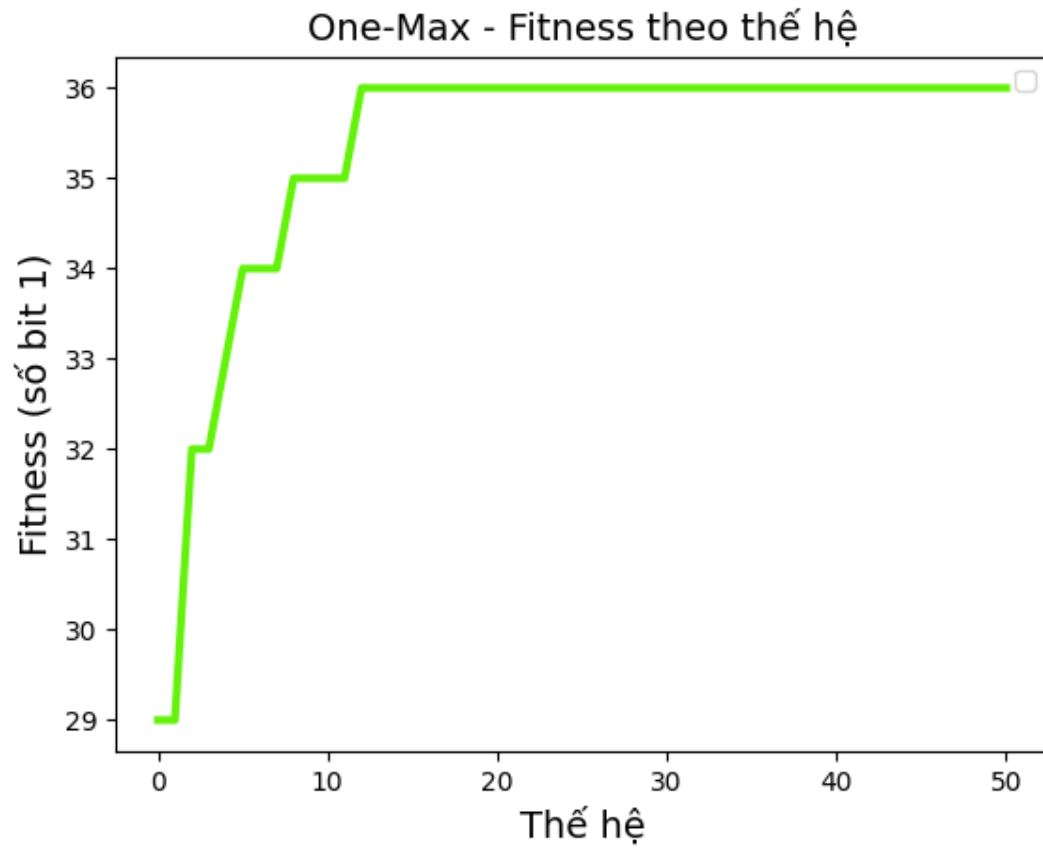
# =====
# 5. Kết quả tốt nhất
# =====
solution, solution_fitness, solution_idx =
ga_instance.best_solution()
print("\nKẾT QUẢ TỐT NHẤT:")
print("Chuỗi nhị phân:", solution)
print(f"Số lượng bit 1: {int(solution_fitness)} / {num_genes}")
print("Index của cá thể tốt nhất trong quần thể:", solution_idx)

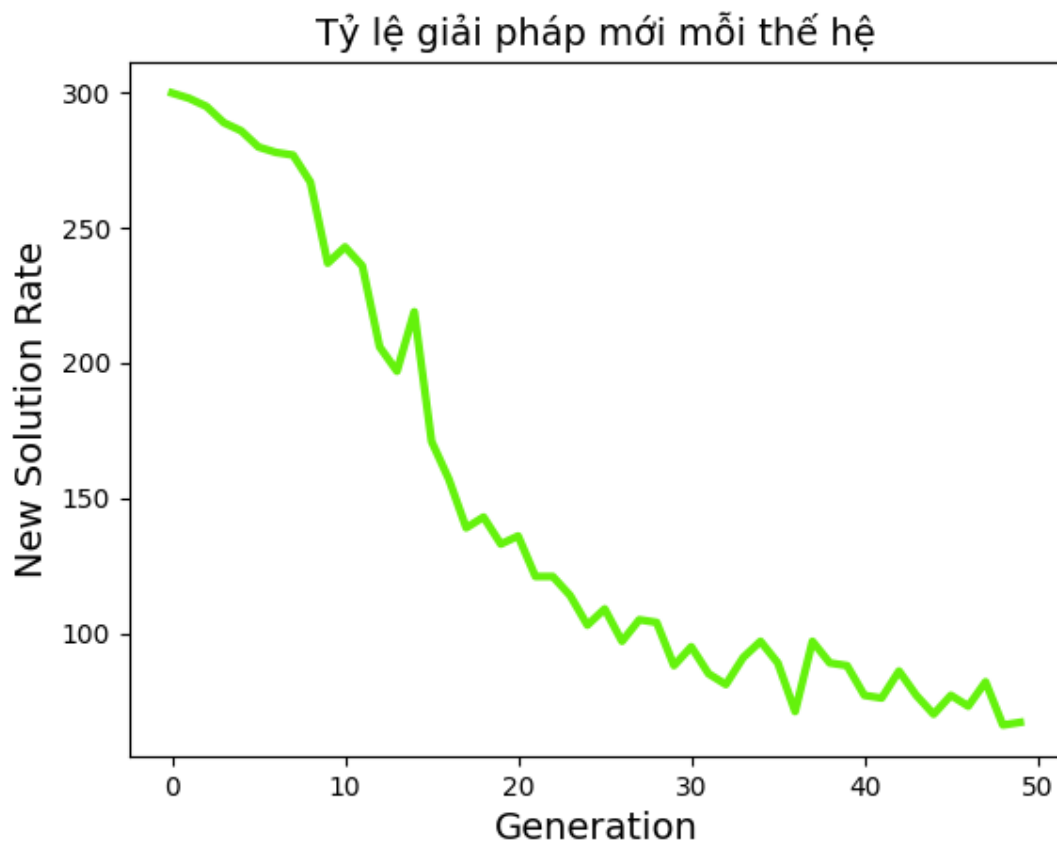
# =====
# 6. Vẽ minh họa (với hàm có sẵn của PyGAD)
# =====
print("\nVẼ BIỂU ĐỒ TIẾN HÓA:")

# Plot 1: Fitness theo thế hệ (tốt nhất + trung bình + tệ nhất)
ga_instance.plot_fitness(title="One-Max - Fitness theo thế hệ",
                        linewidth=3,
                        xlabel="Thế hệ",
                        ylabel="Fitness (số bit 1)",
                        font_size=14)

# Plot 2: Tỷ lệ giải pháp mới được khám phá mỗi thế hệ
ga_instance.plot_new_solution_rate(title="Tỷ lệ giải pháp mới mỗi
thế hệ")

```





Bài tập 2: Tiến hóa chuỗi ký tự (Target Phrase)

Mô tả: Tiến hóa một chuỗi ký tự ngẫu nhiên để trở thành câu đích “GA Xin chào CTU”.

Gợi ý:

- Khởi tạo kích thước quần thể và tham số giống bài 1.
- Tạo cá thể: danh sách ký tự ngẫu nhiên (ASCII 32-126), độ dài bằng target.
- Fitness: số vị trí ký tự trùng với target.
- Chọn lọc tournament giữ nguyên.
- Lai chéo giữ nguyên (vì là list).
- Đột biến: thay ký tự ngẫu nhiên thay vì lật bit.
- Vòng lặp giải thuật và in kết quả.

```
import random
import string
import matplotlib.pyplot as plt
from deap import base, creator, tools, algorithms

TARGET = "GA Xin chào CTU"
```

```

LENGTH = len(TARGET)

creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

toolbox = base.Toolbox()
toolbox.register("attr_char", random.choice, string.ascii_letters +
string.digits + " ,.!")
toolbox.register("individual", tools.initRepeat, creator.Individual,
toolbox.attr_char, LENGTH)
toolbox.register("population", tools.initRepeat, list,
toolbox.individual)

def evaluate(individual):
    return sum(a == b for a, b in zip(individual, TARGET)),

toolbox.register("evaluate", evaluate)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutShuffleIndexes, indpb=0.05)
toolbox.register("select", tools.selTournament, tournsize=3)

def main():
    pop = toolbox.population(n=500)
    hof = tools.HallOfFame(1)
    stats = tools.Statistics(lambda ind: ind.fitness.values)
    stats.register("sum", sum)

    logbook = tools.Logbook()
    logbook.header = ["gen", "sum", "max"]
    pop, logbook = algorithms.eaSimple(pop, toolbox, cxpb=0.7,
mutpb=0.3,
                                ngen=80, halloffame=hof,
verbose=False)

    # Thêm stats để theo dõi
    stats = tools.Statistics(key=lambda ind: ind.fitness.values[0])
    stats.register("sum", lambda hof: sum(i.fitness.values[0] for
i in hof))
    stats.register("max", lambda hof: max(i.fitness.values[0] for
i in hof))

```

```

    print("Kết quả tốt nhất:", "".join(hof[0]))
    print("Fitness:", hof[0].fitness.values[0])
    return hof

if __name__ == "__main__":
    best = main()

```

```

PS C:\Users\nbdie> & C:/python313/python.exe
Kết quả tốt nhất: GA Xin chao CTU
Fitness: 15.0

```

```

import pygad
import numpy as np
import random
import string
import matplotlib.pyplot as plt

# =====
# 1. Cấu hình bài toán
# =====
TARGET = "GA xin chao CTU"
TARGET_LENGTH = len(TARGET)

# Tập ký tự có thể dùng
CHAR_SET = string.ascii_letters + string.digits + " .,-!?"

# =====
# 2. Hàm fitness
# =====
def fitness_func(ga_instance, solution, solution_idx):
    """
    Fitness = số ký tự trùng đúng vị trí với TARGET.
    Trả về float để tránh vấn đề type.
    """
    # solution là numpy array kiểu int (index trong CHAR_SET)
    generated_str = ''.join(CHAR_SET[int(gene)] for gene in
solution)

```

```

    matches = sum(1 for a, b in zip(generated_str, TARGET) if a ==
b)

    # Xử lý an toàn: tránh NaN/inf
    if not np.isfinite(matches):
        return -1e10

    return float(matches)

# =====
# 3. Khởi tạo PyGAD instance
# =====
ga_instance = pygad.GA(
    num_generations=120,          # Số thế hệ (tương đương DEAP)
    num_parents_mating=50,        # Số cha mẹ tham gia lai tạo
    fitness_func=fitness_func,
    sol_per_pop=400,              # Kích thước quần thể (lớn hơn
để hội tụ tốt)
    num_genes=TARGET_LENGTH,      # Số gene = độ dài chuỗi
    save_best_solutions=True,      # Lưu giải pháp tốt nhất của mỗi
thế hệ để vẽ biểu đồ sau này (tồn bộ nhớ hơn)
    save_solutions=True,

    # Gene là chỉ số trong CHAR_SET (0 đến len(CHAR_SET)-1)
    gene_type=int,
    init_range_low=0,
    init_range_high=len(CHAR_SET),

    # Parent selection: steady-state hoặc tournament
    parent_selection_type="sss",    # Hoặc "tournament"

    # Giữ lại một phần elite
    keep_parents=5,

    # Crossover: single-point (phù hợp cho chuỗi)
    crossover_type="single_point",

    # Mutation: random → thay đổi gene thành chỉ số khác
    mutation_type="random",
    mutation_percent_genes=12,      # ~12% gene bị đột biến mỗi cá
thể

```



```

)

# =====
# 4. Chạy GA
# =====
print(f"Bắt đầu tiến hóa chuỗi mục tiêu: '{TARGET}'")
print(f"Độ dài: {TARGET_LENGTH} | Quần thể:
{ga_instance.sol_per_pop} | Thế hệ:
{ga_instance.num_generations}\n")

ga_instance.run()

# =====
# 5. Kết quả tốt nhất
# =====
solution, solution_fitness, idx = ga_instance.best_solution()
best_string = ''.join(CHAR_SET[int(gene)] for gene in solution)

print("\n" + "="*60)
print("KẾT QUẢ TỐT NHẤT SAU KHI TIẾN HÓA:")
print(f"Chuỗi tìm được : '{best_string}'")
print(f"Fitness      : {int(solution_fitness)} / {TARGET_LENGTH}")
print(f"Độ chính xác   : {solution_fitness / TARGET_LENGTH *
100:.1f}%")
print("="*60)

# =====
# 6. Vẽ kết quả (Với hàm của PyGAD)
# =====
print("\nVẼ BIỂU ĐỒ TIẾN TRÌNH:")

# Plot 1: Fitness theo thế hệ (tốt nhất + trung bình + tệ nhất nếu
có)
ga_instance.plot_fitness(
    title="Tiến hóa chuỗi: 'GA xin chào CTU' - Fitness theo thế hệ",
    linewidth=2.5,
    xlabel="Thế hệ",
    ylabel="Fitness (số ký tự đúng)",
    font_size=13
)

```

```
# Plot 2: Tỷ lệ giải pháp mới được khám phá mỗi thế hệ
ga_instance.plot_new_solution_rate(
    title="Tỷ lệ giải pháp mới được tạo ra mỗi thế hệ"
)
```

Nên khởi tạo biến `save_solutions = True` trong GA để lưu kết quả để vẽ

```
C:\python313\Lib\site-packages\pygad\pygad.py:1158: UserWarning: Use the 'save_solutions' parameter with caution as it may cause memory overflow when either the number of generations, number of genes, or number of solutions in population is large.
  warnings.warn("Use the 'save_solutions' parameter with caution as it may cause memory overflow when either the number of generations, number of genes, or number of solutions in population is large.")
```

Bài 3: Bài toán Balo 0/1 (Knapsack)

Mô tả: Chọn đồ vật để tối đa giá trị mà không vượt quá trọng lượng balo (capacity = 50).

Dữ liệu: (weight, value)

```
items = [(10, 60), (20, 100), (15, 80), (25, 120), (5, 30), (12, 70), (8, 50)]
```

Gợi ý:

- Giữ tham số quần thể.
- Tạo cá thể: danh sách nhị phân (0/1) dài bằng số đồ vật.
- Fitness: tổng giá trị nếu tổng trọng lượng $\leq$ capacity, ngược lại trừ nặng (penalty).
- Chọn lọc, lai chéo, đột biến giống bài 1.
- Vòng lặp và in kết quả.

Bài 4: Bài toán Người bán hàng du lịch (TSP) với 5 thành phố

Mô tả: Tìm đường đi ngắn nhất qua 5 thành phố (permutation). Khoảng cách được cho trước (hoặc ngẫu nhiên).

Hướng dẫn từng bước:

- Giữ tham số về quần thể.
- Tạo cá thể: hoán vị ngẫu nhiên của 5 thành phố (0-4).
- Fitness: âm tổng khoảng cách (càng âm càng tốt vì muốn ngắn).
- Chọn lọc tournament.
- Lai chéo Order Crossover (OX) – copy đoạn nhỏ gen rồi điền phần còn lại theo thứ tự.
- Đột biến: hoán đổi 2 vị trí ngẫu nhiên.

- Vòng lặp và in đường đi tốt nhất.

Bài 5 : Sử dụng GA để chơi Flappy Bird tự động

Bài 6 : Thiết kế trò chơi như Mewgenics (lai ghép các mèo con đi thám hiểm – đánh quái)